



UNIVERSIDADE FEDERAL DO CEARÁ CAMPUS DE CRATEÚS

Auditoria e Segurança da Informação - Exercício 2 Professor: Anderson Almada

Nome: _____

Data: 08/10/2026

PARTE I - Preparando o Ambiente

1. Verifique se o python está instalado
`python3 --version`
2. Crie a pasta
`mkdir pratica-criptografia`
`cd pratica-criptografia`
3. Crie dois arquivos na pasta
`bob.py`
`alice.py`
4. Crie o ambiente virtual
`python3 -m venv venv`
`source venv/bin/activate`
5. Instale as dependências
`pip install flask cryptography requests`

PARTE II - Bob gera as chaves RSA e disponibiliza a chave pública

1. Insira esse código no arquivo bob.py

```
from flask import Flask, request  
  
from cryptography.hazmat.primitives.asymmetric import rsa  
from cryptography.hazmat.primitives.asymmetric import padding
```

```

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

import pickle

app = Flask(__name__)

# Bob gera suas chaves RSA
chave_privada_bob = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

chave_publica_bob = chave_privada_bob.public_key()

print("Bob gerou suas chaves RSA.")

app.run(
    host="127.0.0.1",
    port=5001
)

```

2. Crie o endpoint:

```

@app.route("/public-key", methods=["GET"])
def enviar_chave_publica():

    chave_publica_bytes = chave_publica_bob.public_bytes(
        encoding=serialization.Encoding.DER,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )

    return chave_publica_bytes

```

3. Execute no terminal:

```
python bob.py
```

PARTE III - Alice obtém a chave pública de Bob

1. Coloque esse código em alice.py

```
from flask import Flask, request

import requests
import pickle

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes
import os

app = Flask(__name__)

URL_BOB = "http://127.0.0.1:5001"

# Alice solicita a chave pública de Bob
resposta = requests.get(
    f"{URL_BOB}/public-key"
)

chave_publica_bob = serialization.load_der_public_key(
    resposta.content
)

print("Alice recebeu a chave pública de Bob.")

app.run(
    host="127.0.0.1",
    port=5002
)
```

2. Execute no terminal:
python alice.py

PARTE IV - Alice recebe a mensagem

1. Adicione esse trecho de código (alice.py):

```
@app.route("/send", methods=["POST"])
def enviar_mensagem():

    dados = request.get_json()

    mensagem = dados["mensagem"]

    print("\nAlice recebeu a mensagem:")
    print(mensagem)

    return "Mensagem recebida por Alice!"
```

2. Em um novo terminal, realize esse teste:

```
curl -X POST http://127.0.0.1:5002/send \
-H "Content-Type: application/json" \
-d '{"mensagem": "Olá Bob! Esta é uma mensagem secreta."}'
```

PARTE V - Alice gera uma chave AES

1. Dentro de send (alice.py):

```
chave_aes = AESGCM.generate_key(
    bit_length=256
)

mensagem_bytes = mensagem.encode()

aes = AESGCM(chave_aes)

nonce = os.urandom(12)

mensagem_cifrada = aes.encrypt(
    nonce,
    mensagem_bytes,
    None
)
```

2. Protege a chave secreta com a chave pública de Bob

```
chave_aes_cifrada = chave_publica_bob.encrypt(
    chave_aes,
    padding.OAEP(
        mgf=padding.MGF1(
            algorithm=hashes.SHA256()
        ),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

3. Cria um objeto com todos os dados:

```
pacote = {
    "nonce": nonce,
    "mensagem_cifrada": mensagem_cifrada,
    "chave_aes_cifrada": chave_aes_cifrada
}
```

4. Serializa e envia:

```
dados = pickle.dumps(pacote)

resposta = requests.post(
    f'{URL_BOB}/message',
    data=dados
)

print("Bob respondeu:", resposta.text)

return "Mensagem enviada por Alice!\n"
```

PARTE VI - Bob recebe o pacote

1. Adicione esse trecho de código (bob.py):

```
@app.route("/message", methods=["POST"])
def receber_mensagem():
```

```
pacote = pickle.loads(request.data)

nonce = pacote["nonce"]
mensagem_cifrada = pacote["mensagem_cifrada"]
chave_aes_cifrada = pacote["chave_aes_cifrada"]

print("\nBob recebeu o pacote.")
```

2. Descripta tudo e visualiza a mensagem:

```
chave_aes = chave_privada_bob.decrypt(
    chave_aes_cifrada,
    padding.OAEP(
        mgf=padding.MGF1(
            algorithm=hashes.SHA256()
        ),
        algorithm=hashes.SHA256(),
        label=None
    )
)

aes = AESGCM(chave_aes)

mensagem = aes.decrypt(
    nonce,
    mensagem_cifrada,
    None
)

print("Mensagem recebida por Bob:")
print(mensagem.decode())

return "Mensagem recebida!\n"
```

PARTE VII - Responda as seguintes questões:

1. Qual é a principal diferença entre criptografia simétrica e criptografia assimétrica?
2. Quais algoritmos criptográficos foram utilizados nesta atividade e qual é a função de cada um?
3. Por que a mensagem é criptografada utilizando AES-GCM em vez de ser criptografada diretamente com RSA?

4. Por que a solução utiliza AES e RSA simultaneamente? Explique o conceito de criptografia híbrida.
5. Quais são as duas chaves geradas por Bob? Qual é a função de cada uma?
6. Qual chave RSA Alice utiliza para proteger a chave AES? Por que essa chave pode ser compartilhada?
7. Qual chave RSA Bob utiliza para recuperar a chave AES? Por que essa chave deve permanecer secreta?
8. Por que Alice não envia diretamente a chave AES para Bob?
9. Qual é a função da chave AES gerada por Alice?
10. Qual é a função do `nonce` utilizado pelo AES-GCM? Ele precisa ser mantido em segredo?
11. O que aconteceria se a mensagem cifrada fosse alterada antes de chegar a Bob?
12. Qual é o tamanho da chave AES utilizada na atividade?
13. Qual é a função do OAEP utilizado na operação RSA?
14. Por que RSA é utilizado para proteger a chave AES e não para criptografar diretamente toda a mensagem?
15. Qual é a função do endpoint `/public-key` criado por Bob?
16. Qual é a função dos endpoints `/send`, de Alice, e `/message`, de Bob?
17. O que é enviado dentro do `pacote` criado por Alice?
18. Qual é a função do `pickle.dumps()` utilizado por Alice e do `pickle.loads()` utilizado por Bob? Eles realizam criptografia?
19. Se um atacante interceptar o pacote enviado por Alice para Bob, mas não possuir a chave privada de Bob, ele conseguirá recuperar a mensagem original? Explique.
20. Explique todo o caminho percorrido pela mensagem, desde o momento em que Alice recebe o texto até o momento em que Bob consegue visualizar a mensagem original. Indique onde são utilizados AES-GCM, RSA, as chaves pública e privada e a comunicação HTTP.